EECE 7205: Fundamentals of Computer Engineering
Fall 2026
In-Class Work: Deleting from a Binary Search Tree

Paulina Skurzak

Case 1: Deleting the Root

1.

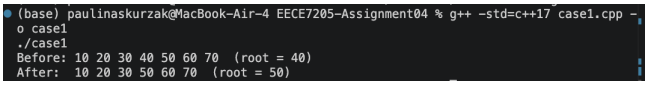
Step	parent	succ
Start	40	60
After loop runs once	60	50

Nodes visited: 60, then 50. The successor is 50, and its parent is 60.

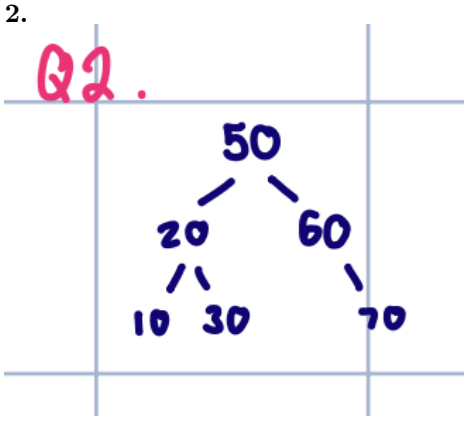
Predicted output (before running):

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 50 60 70   (root = 50)
```

Actual output:



2.



Inorder: 10 20 30 50 60 70
Preorder: 50 20 10 30 60 70

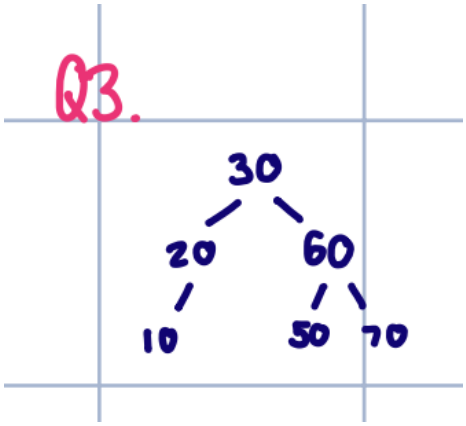
The inorder is still increasing, so B7 still holds.

3.

I changed the code to look for the predecessor instead, going left once then right as far as it can.

```
Node* parent = root;
Node* pred = root->left;                // go left once (20)
while (pred->right != nullptr)          // then right as far
    as possible
{
    parent = pred;
    pred = pred->right;                 // pred = 30, parent
        = 20
}
root->data = pred->data;                // root now holds 30
```

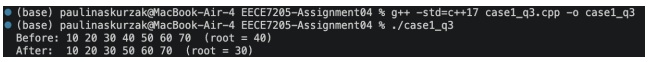
```
if (parent == root)
    parent->left = pred->left;
else
    parent->right = pred->left;         // 20->right = NULL
delete pred;
```



Predicted output (before running):

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 50 60 70   (root = 30)
```

Actual output:

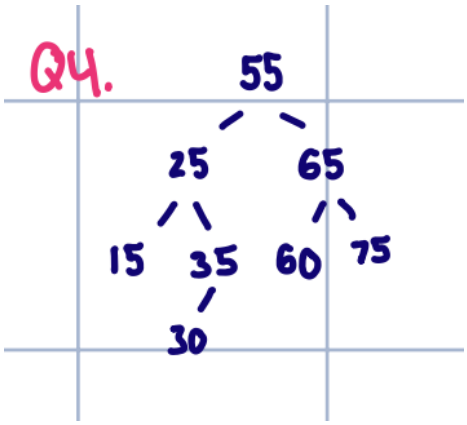


That gave 30 as the new root, with 20 remaining as its left child. The right pointer of 20 now points to NULL because 30 was removed from its original position. The inorder traversal came out as 10 20 30 50 60 70, the same as before. This makes sense because either the inorder successor or predecessor can replace the deleted root while preserving the BST property. The tree shape changes, but the inorder sequence stays the same.

4.

succ starts at 65, moves to 55 since 65 has a left child. 55 has no left child so it stops there. Successor is 55, parent is 65.

This needs rule B4 instead of B3, since 55 still has a right child (60). Using B3 would have deleted 60 along with it.



Predicted output (before running):

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 25 30 35 55 60 65 75   (root = 55)
```

Actual output:

```
Open file in editor (cmd + click)
./case1_q4
Before: 15 25 30 35 45 55 60 65 75  (root = 45)
After:  15 25 30 35 55 60 65 75  (root = 55)
```

Inorder: 15 25 30 35 55 60 65 75

Case 2: Deleting a Leaf

1.

Step	Comparison	Direction
40	10 ¡ 40	left
20	10 ¡ 20	left
10	10 = 10	found

Parent is 20. 10 is 20's left child.

Predicted output (before running):

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  20 30 40 50 60 70   (root = 40)
```

Actual output:

```
(base) paulinaskurzak@MacBook-Air-4 EECE7205-Assignment04 % g++ -std=c++17 case2.cpp -o case2
./case2
Before: 10 20 30 40 50 60 70  (root = 40)
After:  20 30 40 50 60 70  (root = 40)
```

2.

Step	Comparison	Direction
40	70 ¡ 40? No	right
60	70 ¡ 60? No	right
70	70 = 70	found

Parent is 60. 70 is 60's right child, so 60-¿right becomes NULL this time.

Predicted output (before running):

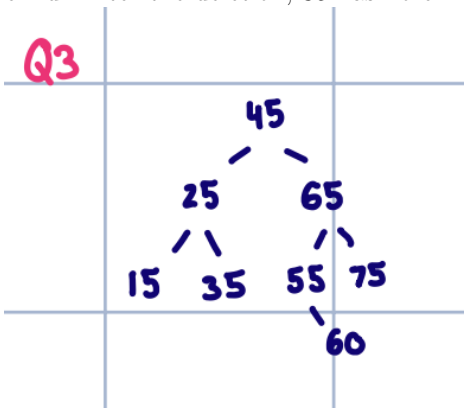
```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 40 50 60   (root = 40)
```

Actual output:

```
(base) paulinaskurzak@MacBook-Air-4 EECE7205-Assignment04 % g++ -std=c++17 case2_q2.cpp -o case2_q2
./case2_q2
Before: 10 20 30 40 50 60 70  (root = 40)
After:  10 20 30 40 50 60  (root = 40)
```

3.

Leaves of tree P: 15, 30, 60, 75. Deleting 30, parent is 35, since 30 is 35's left child. After the deletion, 35 has no children left, so 35 becomes a leaf.



Predicted output (before running):

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 25 35 45 55 60 65 75   (root = 45)
```

Actual output:

```
(base) paulinaskurzak@MacBook-Air-4 EECE7205-Assignment04 % g++ -std=c++17 case2_q3.cpp -o case2_q3
./case2_q3
Before: 15 25 30 35 45 55 60 65 75  (root = 45)
After:  15 25 35 45 55 60 65 75  (root = 45)
```

4.

Worst case is h plus 1 comparisons, one per level root to deepest leaf. Tree T has h equal to 2. A tree built by inserting 10, 20, 30, 40, 50, 60, 70 in order just keeps going right, giving h equal to 6. So deleting a leaf is fast on a balanced tree like T, but much slower on a skewed one like the sorted-insert tree.

Case 3: Deleting a Node with One Child

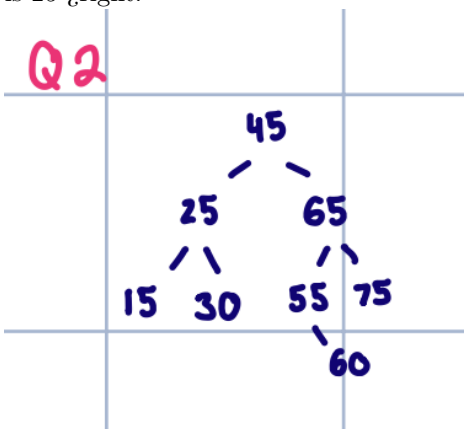
1.

In tree P, two nodes have exactly one child, 35 has one child, 30, on its left. 55 has one child, 60, on its right.

2.

Step	Comparison	Direction
45	35 ¡ 45	left
25	35 ¿ 25	right
35	35 = 35	found

Deleting 35. Parent is 25, child that moves up is 30. The pointer that changes is 25-¿right.



Inorder: 15 25 30 45 55 60 65 75

Predicted output (before running):

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 25 30 45 55 60 65 75   (root = 45)
```

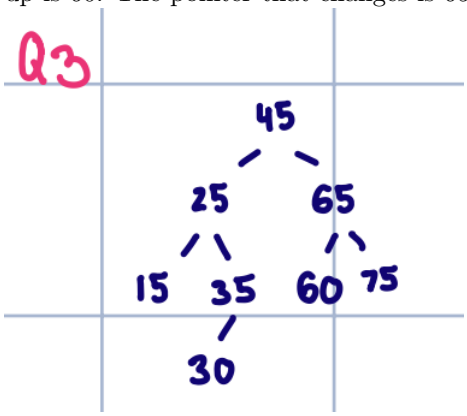
Actual output:

```
(base) paulinaskurzak@MacBook-Air-4 EECE7205-Assignment04 % g++ -std=c++17 case3_q2.cp
p -o case3_q2
./case3_q2
Before: 15 25 30 35 45 55 60 65 75  (root = 45)
After:  15 25 30 45 55 60 65 75  (root = 45)
```

3.

Step	Comparison	Direction
45	55 ¿ 45	right
65	55 ¡ 65	left
55	55 = 55	found

Starting again from the original P, deleting 55. Parent is 65, child that moves up is 60. The pointer that changes is 65-¿left.



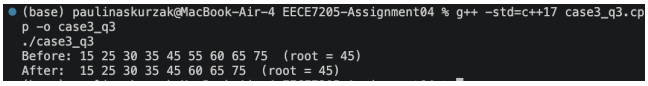
Inorder: 15 25 30 35 45 60 65 75

It uses parent-¿left this time instead of parent-¿right because 55 really is 65's left child.

Predicted output (before running):

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 25 30 35 45 60 65 75   (root = 45)
```

Actual output:



4.
Inserting 10, 20, 30 in that order gives a chain, 10 at root with 20 as its right child and 30 as 20's right child. Deleting root 10, which has one child, rule B6 says the root pointer itself must change. New root is 20.

If case3.cpp is used unchanged, this breaks, because parent starts as nullptr and the loop never runs its body, since cur already equals the key on the first check. Then parent-¿left crashes, since there's no node there to check.

Case 4: Deleting a Node with Two Children

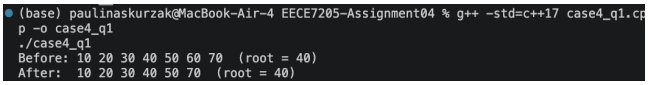
1.
Searching for 60 finds it directly, so cur = 60. succ starts at 70, parent at 60. 70 has no left child so the loop never runs.

Since parent equals cur here, the first branch runs: parent-¿right = succ-¿right. 70 has no right child, so 60's old right pointer becomes NULL. The node that held 60 now holds 70, keeping 50 as its left child.

Predicted output (before running):

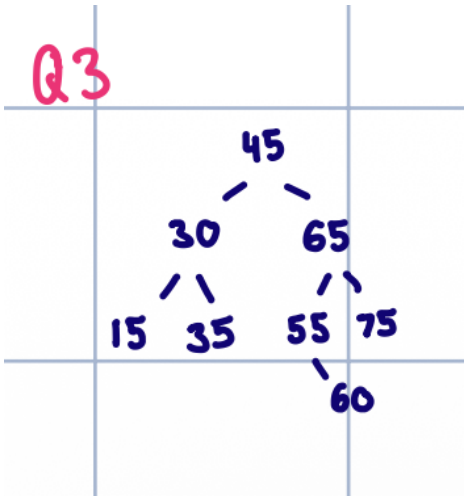
```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 40 50 70   (root = 40)
```

Actual output:



2.
The successor is found by going right once then left as far as possible. Going left as far as possible means you stop exactly when there's no left child left to follow, so the successor can never have one. If it did, step 4 would be wrong, since the code only reattaches succ-¿right, and that left subtree would never get reconnected to anything.

3.
Deleting 25. Successor is 30, parent is 35. Since parent does not equal cur here, the pointer that changes is 35-¿left.



Inorder: 15 30 35 45 55 60 65 75

Predicted output (before running):

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 30 35 45 55 60 65 75   (root = 45)
```

Actual output:

```
(base) paulinaskurzak@MacBook-Air-4 EECE7205-Assignment04 % g++ -std=c++17 case4_q3.cp
p -o case4_q3
./case4_q3
Before: 15 25 30 35 45 55 60 65 75  (root = 45)
After:  15 30 35 45 55 60 65 75  (root = 45)
```

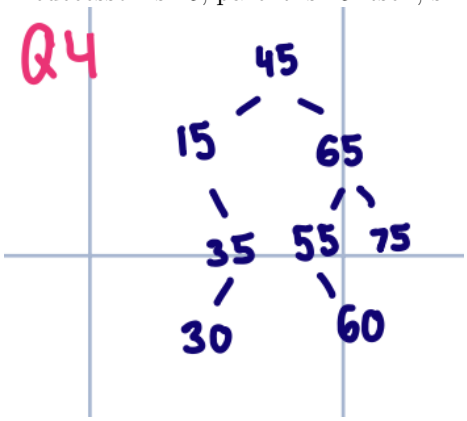
4.

Deleting 25 again, using the predecessor instead.

```
Node* cur = root;
int key = 25;
while (cur != nullptr && cur->data != key)
{
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
Node* parent = cur;
Node* pred = cur->left;                  // 15
while (pred->right != nullptr)
{
    parent = pred;
    pred = pred->right;
}
cur->data = pred->data;                  // node that held 25
     now holds 15
if (parent == cur)
    parent->left = pred->left;           // 15 was cur's left
         child
else
```

```
    parent->right = pred->left;
delete pred;
```

Predecessor is 15, parent is 25 itself, since the loop never moves away from cur.



Inorder: 15 30 35 45 55 60 65 75, same as question 3. Tree shape is different, since 15 is now the left child's root instead of 30, but both are valid binary search trees since the inorder sequence is identical either way.

Predicted output (before running):

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 30 35 45 55 60 65 75   (root = 45)
```

Actual output:

```
(base) paulinaskurzak@MacBook-Air-4 EECE7205-Assignment04 % g++ -std=c++17 case4_q4.cp
p -o case4_q4
./case4_q4
Before: 15 25 30 35 45 55 60 65 75  (root = 45)
After:  15 30 35 45 55 60 65 75  (root = 45)
```

Case 5: The Key Is Not in the Tree

1.

Searching for 25, 25 less than 40, go left. 25 greater than 20, go right. 25 less than 30, go left. 3 comparisons, stops at 30's left null link.

2.

Key	Comparisons	Stops at
5	3	10's left
45	3	50's left
75	3	70's right

All four keys take exactly 3 comparisons. Tree T is perfectly balanced with height 2, so every empty spot sits at the same depth, and the worst case is always height plus 1.

3.

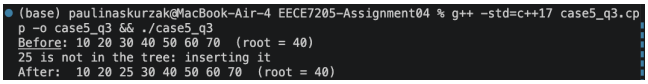
I changed the code to remember the parent during the search, so if the key is not found, a new node can be attached at the right spot.

```
Node* parent = nullptr;
Node* cur = root;
int key = 25;
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
if (cur == nullptr)
{
    cout << key << " is not in the tree: inserting it" <<
        endl;
    Node* newNode = new Node(key);
    if (key < parent->data) parent->left = newNode;
    else                    parent->right = newNode;
}
```

Predicted output (before running):

```
Before: 10 20 30 40 50 60 70   (root = 40)
25 is not in the tree: inserting it
After:  10 20 25 30 40 50 60 70   (root = 40)
```

Actual output:



4.
I added a check for an empty tree before printing root-¿data, since that would crash otherwise.

```
if (root == nullptr)
    cout << "the tree is empty" << endl;
else
{
    cout << "After:  ";
    printInorder(root);
    cout << " (root = " << root->data << ")" << endl;
}
```

Rule B2 is the closest match, since it addresses a key that is not found in the tree. Checking whether the root is NULL before printing is an additional safeguard needed to handle an empty tree safely.

Predicted output (before running):

```
the tree is empty
```

Actual output:



Bonus: One Function for Every Case

I combined all five cases into one recursive function that always returns the possibly new root.

```
Node* deleteNode(Node* root, int key)
{
    // not found, the search reached NULL
    if (root == nullptr)
        return root;

    // not found yet, keep searching, left or right
    if (key < root->data)
    {
        root->left = deleteNode(root->left, key);
        return root;
    }
    if (key > root->data)
    {
        root->right = deleteNode(root->right, key);
        return root;
    }

    // found it, root->data == key

    // leaf, or only a right child
    if (root->left == nullptr)
    {
        Node* onlyChild = root->right;
        delete root;
        return onlyChild;
    }
    // only a left child
    if (root->right == nullptr)
    {
        Node* onlyChild = root->left;
        delete root;
        return onlyChild;
    }

    // two children, find the smallest value in the right
       subtree
    Node* succParent = root;
    Node* succ = root->right;
    while (succ->left != nullptr)
    {
        succParent = succ;
        succ = succ->left;
    }
    root->data = succ->data;
```

```
    if (succParent == root)
        succParent->right = succ->right;
    else
        succParent->left = succ->right;
    delete succ;
    return root;
}
```

Deleting the root doesn't need its own branch, since whatever the function returns becomes the caller's new root.

Testing on tree T, deleting 40, 10, 60, and 25 one after another:

```
Before: 10 20 30 40 50 60 70   (root = 40)
After deleting 40: 10 20 30 50 60 70   (root = 50)
After deleting 10: 20 30 50 60 70   (root = 50)
After deleting 60: 20 30 50 70   (root = 50)
After deleting 25: 20 30 50 70   (root = 50)
```

Actual output:

```
g++ -std=c++17 bonus.cpp -o bonus
./bonus
Before: 10 20 30 40 50 60 70  (root = 40)
After deleting 40: 10 20 30 50 60 70  (root = 50)
After deleting 10: 20 30 50 60 70  (root = 50)
After deleting 60: 20 30 50 70  (root = 50)
After deleting 25: 20 30 50 70  (root = 50)
```

Deleting 40 hit the two children and root cases together. Deleting 10 was a plain leaf. Deleting 60 only had one child left at that point, since its left child was already removed when 40 was deleted. Deleting 25 did nothing, since it was never in the tree.

Time complexity: $\Theta(h)$. Each call does constant work before either going one level deeper, which happens at most h times, or doing the actual deletion. The two children case just adds one more walk down the right subtree for the successor, still bounded by h.